WEEK – 9

Experiment No. 9: Packages in Java

Title

Roman to Integer Conversion Using Packages

Objective / Aim

To write a Java program to define, create, and access a user-defined package and demonstrate importing packages by converting a Roman numeral into its corresponding integer value using the `romanToInteger()` method.

Problem Statement

Create a package named `number` and define a class named `Roman` inside it. Implement the `romanToInteger()` method to convert a Roman numeral into its corresponding integer value.

Import the `Roman` class into another class and call the `romanToInteger()` method to display the converted integer.

Input: `"LVIII"`

Output: `58`

Explanation:

L = 50, V = 5, III = 3.

Therefore, LVIII = 50 + 5 + 3 = 58.

Concepts Used

Concept	Description
Package	A package is used to group related classes and interfaces.
User-Defined Package	A package created by the programmer.
`package` Keyword	Used to declare a package in Java.

Concept	Description
`import` Keyword	Used to access classes from another package.
Class	A blueprint for creating objects and defining methods.
Method	The `romanToInteger()` method converts a Roman numeral into an integer.
String	Used to store the Roman numeral.
Switch Statement	Used to determine the integer value of each Roman symbol.
Conditional Statement	Used to determine whether a symbol's value should be added or subtracted.

Classes Used

1. **Roman:** A class defined inside the `number` package that implements the `romanToInteger()` method.
2. **RomanDemo:** The main class that imports the `Roman` class, calls the method, and displays the result.

Algorithm / Logic

1. Create a package named `number`.
2. Define a public class named `Roman` inside the package.
3. Implement the `romanToInteger()` method that accepts a Roman numeral as a string.
4. Define the integer values of the Roman symbols I, V, X, L, C, D, and M.
5. Traverse the Roman numeral from left to right.
6. Get the integer value of the current Roman symbol.
7. If the current symbol has a smaller value than the next symbol, subtract its value from the total.
8. Otherwise, add its value to the total.
9. Repeat the process until all symbols have been processed.
10. Return the calculated integer value.
11. Create another class named `RomanDemo`.
12. Import the `Roman` class using the `import number.Roman;` statement.
13. Create an object of the `Roman` class.
14. Pass the Roman numeral to the `romanToInteger()` method.
15. Display the corresponding integer value.

Source Code

File 1: Roman.java

```
package number;

public class Roman {

    public int romanToInteger(String s) {
        int total = 0;

        for (int i = 0; i < s.length(); i++) {
            int current = value(s.charAt(i));

            if (i + 1 < s.length() &&
                current < value(s.charAt(i + 1))) {
                total -= current;
            } else {
                total += current;
            }
        }

        return total;
    }

    private int value(char ch) {
        switch (ch) {
            case 'I': return 1;
            case 'V': return 5;
            case 'X': return 10;
            case 'L': return 50;
            case 'C': return 100;
            case 'D': return 500;
            case 'M': return 1000;

            default:
                throw new IllegalArgumentException(
                    "Invalid Roman numeral character: " + ch
                );
        }
    }
}
```

File 2: RomanDemo.java

```
import number.Roman;

public class RomanDemo {

    public static void main(String[] args) {

        Roman obj = new Roman();

        String roman = "LVIII";
```

```
        int result = obj.romanToInteger(roman);

        System.out.println("Roman Numeral: " + roman);
        System.out.println("Integer Value: " + result);
    }
}
```

Package Structure

The program contains the following files:

```
Project
|
|-- number
|   |-- Roman.java
|
|-- RomanDemo.java
```

The `Roman.java` file is placed inside the `number` folder because it belongs to the `number` package. The `RomanDemo.java` file is placed in the parent project directory.

Compilation and Execution

Step 1: Compile the program

Open the terminal in the project directory and execute the following command:

```
javac -d . number/Roman.java RomanDemo.java
```

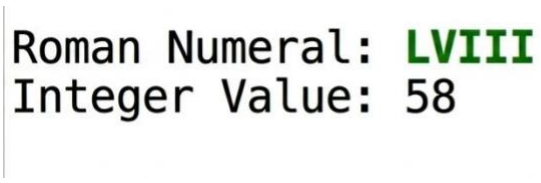
Step 2: Run the program

Execute the following command:

```
java RomanDemo
```

Sample Output

```
Roman Numeral: LVIII
Integer Value: 58
```



Test Cases

Test Case 1: Basic Roman Numeral

Input: `LVIII`

Expected Output: `58`

Explanation: L = 50, V = 5, III = 3.

Result: 50 + 5 + 3 = 58.

Test Case 2: Roman Numeral with Subtraction

Input: `IV`

Expected Output: `4`

Explanation: I = 1 and V = 5. Since I appears before V, subtract 1 from 5.

Result: 5 - 1 = 4.

Test Case 3: Roman Numeral Representing 9

Input: `IX`

Expected Output: `9`

Explanation: I = 1 and X = 10. Since I appears before X, subtract 1 from 10.

Result: 10 - 1 = 9.

Test Case 4: Roman Numeral Representing 40

Input: `XL`

Expected Output: `40`

Explanation: X = 10 and L = 50. Since X appears before L, subtract 10 from 50.

Result: 50 - 10 = 40.

Test Case 5: Roman Numeral Representing 90

Input: `XC`

Expected Output: `90`

Explanation: X = 10 and C = 100. Since X appears before C, subtract 10 from 100.

Result: 100 - 10 = 90.

Test Case 6: Roman Numeral Representing 400

Input: `CD`

Expected Output: `400`

Explanation: C = 100 and D = 500. Since C appears before D, subtract 100 from 500.

Result: 500 - 100 = 400.

Test Case 7: Roman Numeral Representing 900

Input: `CM`

Expected Output: `900`

Explanation: C = 100 and M = 1000. Since C appears before M, subtract 100 from 1000.

Result: 1000 - 100 = 900.

Test Case 8: Roman Numeral Representing 2024

Input: `MMXXIV`

Expected Output: `2024`

Explanation: MM = 2000, XX = 20, and IV = 4.

Result: 2000 + 20 + 4 = 2024.

Test Case Summary

Test Case	Input	Expected Output
1	LVIII	58
2	IV	4
3	IX	9
4	XL	40
5	XC	90

Test Case	Input	Expected Output
6	CD	400
7	CM	900
8	MMXXIV	2024

Note: These test cases represent the method's expected results for valid Roman numeral strings. The provided `RomanDemo` program uses `"LVIII"` as its fixed input. To test the other cases, change the value assigned to the `roman` variable in `RomanDemo.java` and run the program again.

Explanation

The program demonstrates the use of a user-defined package in Java.

The `Roman` class is defined inside the `number` package using the following statement:

```
package number;
```

The `RomanDemo` class imports the `Roman` class using:

```
import number.Roman;
```

An object of the `Roman` class is created, and the `romanToInteger()` method is called to convert the Roman numeral into an integer.

The method uses a `switch` statement to identify the value of each Roman symbol. If a symbol has a smaller value than the symbol that follows it, its value is subtracted. Otherwise, its value is added to the total.

For example, for the Roman numeral `LVIII`:

- L = 50
- V = 5
- I = 1
- I = 1
- I = 1

Therefore, the total is:

50 + 5 + 1 + 1 + 1 = 58.

The program displays the result as `58`.

Result

Thus, the Java program successfully demonstrates defining, creating, and accessing a user-defined package and importing a class from that package to convert Roman numerals into their corresponding integer values. The program produces the expected output for the given input `LVIII`, which is `58`.

Conclusion

The program demonstrates how packages are used to organize classes in Java. It also demonstrates how to create a user-defined package, import a class from that package, and access its public methods from another class. The `romanToInteger()` method successfully converts valid Roman numeral strings into their corresponding integer values.